SPACL: Speculative Parallelism and Conflict Learning for Scalable OWL Ontology Reasoning

Anusorn Chaikaew[a,1], Varin Chouvatut[a,2], Ekkarat Boonchieng[a,3,*]

[a]*Chiang Mai University, Department of Computer Science, Faculty of Science, Chiang Mai, 50200, Thailand*

Abstract

We present SPACL (Speculative Parallel Tableaux with Adaptive Conflict Learning), an open-source ALC/SHOIQ reasoner that combines speculative parallelism with conflict-driven learning. Throughout this manuscript, *SPACL* denotes both the algorithm and the public reference implementation. Relative to mainstream open OWL2 DL pipelines discussed in Section 2, SPACL emphasizes the combination of work-stealing speculative execution and reusable nogood information in one implementation path.

Our key contributions are: (1) a speculative parallel tableaux framework with work-stealing scheduling; in the current prototype this is realized via branch-constrained work items evaluated in parallel; (2) an adaptive threshold mechanism that selects sequential or parallel execution based on estimated complexity, yielding 18–73× speedup over naive always-parallel execution on small ontologies; (3) thread-local nogood caching that keeps most repeated nogood lookups on the local hot path in dedicated inconsistent-workload ablations (82–95% local cache-hit rates); (4) a global thread pool optimization that reduces overhead by 2.1×; and (5) a Rust implementation with parser support for RDF/XML, OWL2 Functional, Manchester, and JSON-LD syntaxes. On synthetic scalability workloads, SPACL reaches up to 4.88× speedup at 10,000 classes while keeping overhead below 2× on smaller ontologies. These synthetic scalability figures are supporting evidence;

*Corresponding author

Email addresses: `anusorn_chaikaew@cmu.ac.th` (Anusorn Chaikaew), `varin.ch@cmu.ac.th` (Varin Chouvatut), `ekkarat.boonchieng@cmu.ac.th` (Ekkarat Boonchieng)

[1]First author

[2]Second author

[3]Corresponding author

the main benchmark basis of the paper is the repeated-run end-to-end ontology evaluation reported below.

In the profile-mixed Standard Workload Suite (7 ontologies; 3 clean runs), per-ontology medians among successful comparable runs place SPACL lowest on 4/7 ontologies; ELK (OWL EL profile) is lowest on the two deepest hierarchy files, and Pellet is lowest on `univ-bench.owl`. In the primary Large Head-to-Head benchmark (doid/go-basic/uberon; 3 clean runs), SPACL has the lowest median wall-clock time on all three tested ontologies among successful comparable runs under a common 900s timeout protocol. A suite-level failure-aware metric (capped expected wall-time, CEW) over comparable runs reports ELK lowest on Standard and SPACL lowest on Large. In an external OWL2Bench core-profile validation (DL/EL/QL/RL), three clean repeated runs show SPACL has the lowest median wall-clock time on 4/4 ontologies with 12/12 successful runs in this harness. Timeout policy is reported explicitly in all benchmark tables (Standard Workload Suite: 300s; Large Head-to-Head: 900s; OWL2Bench core profile suite: 300s). This improvement is driven by a structural RDF/XML parser and optimized materialization path that reuses the same ALC/SHOIQ semantic handlers as the baseline path; stage-level A/B runs show parse-only reductions of 96.96–97.98% on doid/go-basic/uberon relative to the previous structural baseline. Head-to-head suites are reported as descriptive repeated-run wall-clock comparisons, not formal significance tests.

Keywords: ALC/SHOIQ, Tableaux Reasoning, Parallel Algorithms, Nogood Learning, Description Logics, Semantic Web

---

1. Introduction

Description Logics (DLs) provide the formal basis of the Semantic Web, and OWL2 DL is the corresponding W3C standard ontology language [1]. Tableaux methods are still widely used for OWL2 DL reasoning because they provide sound and complete procedures for expressive TBox and ABox tasks [2]. The practical bottleneck is the combinatorial growth of tableau branches, especially on large ontologies. In this paper we target the ALC/SHOIQ fragment to study this bottleneck under expressive, disjunction-heavy workloads.

Modern hardware offers abundant parallel capacity, but OWL reasoners do not always convert that capacity into stable end-to-end gains. Some strong parallel systems are integrated as external binaries in practical harnesses (e.g., Konclude) [3], which can limit instrumentation parity and failure diagnosis in cross-system

experiments. Many widely used open implementations (e.g., Pellet/Openllet and HermiT) remain primarily sequential in their core exploration loop [4, 5, 6]. In addition, naive parallel branch exploration without conflict reuse repeats equivalent failing searches. These constraints also motivate our longer-term goal: a reasoning core for ontology transaction validation in distributed, permissioned application settings. This paper focuses on the reasoning and parsing foundations required for that deployment path.

1.1. Research Questions

This work addresses the following research questions:

RQ1: Can speculative parallel tableaux with conflict-driven nogood learning improve OWL2 DL consistency checking performance?

RQ2: Can adaptive parallelism reduce overhead on small workloads while preserving scalability on larger workloads?

RQ3: For large RDF/XML ontologies, which stages dominate end-to-end runtime, and how much can structural parsing/materialization optimization reduce this bottleneck while retaining implementation-aligned semantic behavior?

1.2. Contributions

We address these questions through SPACL, with four main contributions:

C1: **Speculative parallel tableaux framework with conflict learning**: SPACL combines work-stealing execution with nogood extraction and reuse. The current prototype realizes speculative exploration via branch-constrained work items generated from selected disjunctions, reducing redundant failure exploration in non-deterministic search (Sections 4.2, 4.3).

C2: **Adaptive overhead control**: We integrate threshold-based strategy selection with a global thread pool, reducing small-workload parallel overhead while preserving speedup at larger scales (Sections 4.4, 6.5).

C3: **Structural RDF/XML pipeline with handler-equivalent materialization**: We implement a structural parsing/materialization path for the evaluated ALC/SHOIQ handling path and substantially reduce parser-dominated runtime on large ontologies (Section 5.5).

C4: **Reproducible end-to-end evaluation**: We provide a containerized benchmark protocol and artifacts for stage-level analysis and multi-reasoner head-to-head comparison (Sections 6.2, 6.9).

In Rust, these design choices translate into measurable gains on our synthetic benchmark groups: up to 4.88× speedup at 10,000 classes, measured per-class processing below 1μs at the largest synthetic point, and under 2× overhead on small ontologies. On the same synthetic scalability workloads, SPACL reaches 26.2 million measured tableau rule applications per second. These synthetic throughput and scaling figures are supporting microbenchmark signals; the main end-to-end ontology claims in this paper come from the repeated-run head-to-head and stage-level evaluations reported later.

1.3. Paper Organization

The remainder of this paper is organized as follows. Section 2 reviews related work in parallel reasoning and conflict learning. Section 4 presents the SPACL algorithm and its components. Section 5 describes our Rust implementation. Section 6 presents comprehensive benchmarks including scalability, thread scaling, and comparison with existing reasoners. Section 7 concludes with limitations and future directions.

2. Related Work

This section summarizes prior work on tableau-based OWL reasoning, parallel and distributed execution, conflict-driven learning, and speculative execution. The goal is to position SPACL against the strongest established techniques used in practical reasoners.

2.1. Tableau-Based Reasoning

Tableau calculi remain a central approach for expressive description logic reasoning. Practical systems such as Pellet and HermiT demonstrated that sound and complete reasoning can be delivered at scale when rule ordering, blocking, and caching are engineered carefully [4, 6, 7]. In particular, the hypertableau line of work reduces non-deterministic branching pressure compared with classical tableau expansion, and individual-reuse techniques reduce model blow-up in challenging ontologies [7].

State-of-the-art systems also combine calculi. Konclude integrates tableau-style reasoning with additional optimization layers and has shown strong performance on large benchmark sets [3, 8, 9]. For tractable fragments, consequence-based reasoners such as ELK provide excellent throughput, illustrating how calculus design strongly influences scalability and implementation trade-offs [10].

2.2. *Parallel Reasoning*

Parallelization has been studied at multiple granularities. Shared-memory classification frameworks distribute satisfiability and subsumption checks across workers and report substantial speedup on multicore systems [11, 12]. Work-stealing style scheduling further improves load balancing when ontology subproblems are uneven [13, 11].

Beyond classification, parallel ABox reasoning has explored dynamic splitting of model-construction tasks with cache reuse strategies to improve throughput on assertion-heavy workloads [14]. Distributed materialization frameworks (for less expressive profiles) use Spark-based execution and data-partitioned pipelines to scale to very large datasets [15, 16, 17]. These results confirm the scalability potential of parallel infrastructures, while also highlighting that end-to-end performance is highly sensitive to reasoning profile, task type, and synchronization overhead.

2.3. *Conflict-Driven Learning*

Conflict-driven learning is a mature optimization paradigm in SAT/CSP, where learned clauses/nogoods prevent repeated exploration of failing search regions [18, 19]. In description logic systems, related ideas appear in more localized forms, such as nogood-aware individual reuse in hypertableau procedures [7].

Hybrid reasoners that couple tableau and saturation procedures also demonstrate the value of reusing derived information to reduce repeated work during classification [8, 9, 3]. However, openly documented integrations of branch-level speculative parallel tableau exploration with adaptive, reusable nogood learning remain limited in mainstream open-source OWL2 DL pipelines.

2.4. *Speculative Execution*

Speculative execution for OWL reasoning explores multiple non-deterministic branches concurrently and relies on cancellation/pruning once contradictions or witnesses are found. Work-stealing provides a practical runtime foundation for this model by balancing branch workloads dynamically [13, 11]. Lock-free and low-contention engineering techniques are also critical for obtaining net gains rather than synchronization-bound slowdowns [10, 12].

The primary systems challenge is overhead control. On easy workloads, naive always-parallel execution can underperform sequential reasoning because scheduling and coordination costs dominate. This motivates adaptive policies that switch between sequential and parallel execution based on estimated branch complexity [11]. A second challenge is conflict sharing: learned nogoods should be reused

aggressively enough to prune search, but not so aggressively that contention erodes parallel benefit [14].

2.5. Summary and Positioning

Prior literature provides strong building blocks: optimized tableau/hypertableau calculi [7, 6], high-throughput parallel classification frameworks [11, 12], distributed materialization engines [15, 16, 17], and mature nogood machinery in constraint solving [19]. SPACL is positioned at their intersection: branch-level speculative parallelism, adaptive overhead control, and practical nogood reuse designed for expressive OWL reasoning workloads. To make this positioning analytically precise, the next section formalizes the decision model, branching behavior, and correctness arguments that underlie the implementation and evaluation.

3. Formal Framework

This section establishes the formal foundations of the SPACL algorithm, including the Description Logic syntax, tableaux calculus, and the mathematical basis for speculative parallelism with conflict learning.

3.1. Description Logic ALC

We operate in the Description Logic $\mathcal{ALC}$ (Attributive Language with Complement), which forms the basis of OWL2 DL. Our implementation extends to $\mathcal{SHOIQ}$ (with transitive roles, role hierarchies, nominals, inverse roles, and qualified cardinality restrictions). The syntax is defined as follows:

Definition 3.1 (ALC Syntax)**.** Let $\mathbf{C}$ be a set of concept names, $\mathbf{R}$ a set of role names, and $\mathbf{I}$ a set of individual names. *Concepts* C, D are defined by the grammar:

$$C, D ::= A \mid \top \mid \bot \mid \neg C \mid C \sqcap D \mid C \sqcup D \mid \exists r.C \mid \forall r.C \tag{1}$$

where $A \in \mathbf{C}$ is an atomic concept and $r \in \mathbf{R}$ is a role.

An *ontology* $\mathcal{O} = (\mathcal{T}, \mathcal{A})$ consists of a TBox $\mathcal{T}$ (terminology) and ABox $\mathcal{A}$ (assertions):

$$\mathcal{T} ::= \{C \sqsubseteq D\} \cup \{C \equiv D\} \tag{2}$$

$$\mathcal{A} ::= \{a : C\} \cup \{(a, b) : r\} \tag{3}$$

3.2. Tableaux Calculus

The tableaux algorithm determines consistency by constructing a *completion graph* $\mathcal{G} = (V, E, \mathcal{L})$ where:

- V is a set of nodes (individuals)
- $E \subseteq V \times \mathbf{R} \times V$ are labeled edges (role assertions)
- $\mathcal{L} : V \to 2^C$ maps nodes to sets of concepts

Definition 3.2 (Clash)**.** A node $x \in V$ contains a *clash* if either:

1. $\{C, \neg C\} \subseteq \mathcal{L}(x)$ for some concept C, or
2. $\bot \in \mathcal{L}(x)$

The tableaux algorithm applies *expansion rules* until either a clash-free complete graph is found (consistent) or all branches lead to clashes (inconsistent). Table 1 summarizes the core $\mathcal{ALC}$ expansion rules used in our formal presentation.

Table 1: Tableaux Expansion Rules for $\mathcal{ALC}$

Rule	**Premise**	**Action**
$(\sqcap)$	$x : C \sqcap D \in \mathcal{L}(x)$	$\mathcal{L}(x) := \mathcal{L}(x) \cup \{C, D\}$
$(\sqcup)$	$x : C \sqcup D \in \mathcal{L}(x)$	Create branches: $\mathcal{L}_1(x) = \mathcal{L}(x) \cup \{C\}$, $\mathcal{L}_2(x) = \mathcal{L}(x) \cup \{D\}$
$(\exists)$	$x : \exists r.C \in \mathcal{L}(x)$	Add new y, $(x, y) : r$, $y : C$
$(\forall)$	$x : \forall r.C \in \mathcal{L}(x)$, $(x, y) : r$	$\mathcal{L}(y) := \mathcal{L}(y) \cup \{C\}$

3.3. Speculative Parallelism Model

Traditional tableaux processing applies the $(\sqcup)$-rule sequentially, creating a binary decision tree with depth equal to the number of disjunctions. SPACL introduces *speculative parallelism*:

Definition 3.3 (Speculative Branch)**.** Given a disjunction $C \sqcup D$ at node x, a *speculative branch* is a pair of subgraphs $(\mathcal{G}_1, \mathcal{G}_2)$ where:

$$\mathcal{G}_1 = (V, E, \mathcal{L}[x \mapsto \mathcal{L}(x) \cup \{C\}]) \tag{4}$$

$$\mathcal{G}_2 = (V, E, \mathcal{L}[x \mapsto \mathcal{L}(x) \cup \{D\}]) \tag{5}$$

Both branches are explored concurrently by different workers.

Heuristic speedup model. Let T_{seq} be sequential runtime and T_{par} speculative-parallel runtime with p workers. A practical lower-bound model is:

$$T_{par} \gtrsim \frac{T_{seq}}{p} \cdot (1 + \alpha \cdot o_{sync}) \tag{6}$$

where α is the synchronized-work fraction and o_{sync} is synchronization overhead. We use this expression as an intuition aid for overhead analysis, not as a formal theorem.

3.4. Nogood Learning

Definition 3.4 (Nogood)**.** A *nogood* is a set of class expressions $\mathcal{N} = \{C_1, \ldots, C_n\}$ that is jointly unsatisfiable under conjunction in the current branch context, i.e., $\bigcap_{i=1}^{n} C_i \sqsubseteq \bot$.

Definition 3.5 (Nogood Subsumption)**.** A nogood $\mathcal{N}$ *subsumes* a branch with constraints Γ if:

$$\mathcal{N} \subseteq \Gamma \tag{7}$$

If $\mathcal{N}$ subsumes Γ, the branch can be immediately pruned.

Algorithm 1 Verified Nogood Extraction (Pruning-Safe)

Require: Contradictory branch constraints Γ_b, ontology $\mathcal{O}$
Ensure: Verified nogood $\mathcal{N}$ for pruning, or $\emptyset$

```
N ← Γ_b
for each constraint expression e ∈ Γ_b do
    if O ∪ (N \ {e}) ⊨ ⊥ then
        N ← N \ {e}                          ▹ Greedy core minimization
    end if
end for
if O ∪ N ⊨ ⊥ then
    return N
else
    return ∅                                 ▹ No pruning-authorized nogood
end if
```

3.5. Adaptive Threshold

The adaptive threshold mechanism uses two implementation-aligned gates to decide between sequential and parallel processing.

Definition 3.6 (Implementation Cost Model)**.** For expression-level scoring, the prototype uses:

$$\text{cost}(\text{ObjectUnionOf}(ops)) = 50 \times |ops| + \sum \text{cost}(op) \tag{8}$$

$$\text{cost}(\text{ObjectIntersectionOf}(ops)) = 25 \times |ops| + \sum \text{cost}(op) \tag{9}$$

$$\text{cost}(\text{ObjectComplementOf}(e)) = 15 + \text{cost}(e) \tag{10}$$

$$\text{cost}(\text{atomic}) = 10 \tag{11}$$

Definition 3.7 (Two-Gate Parallel Enable Rule)**.** Let $B(\mathcal{O})$ be branch-count estimate:

$$B(\mathcal{O}) = \max\left(1,\ |\mathcal{D}| + 2 \times |\text{DisjointAxioms}| + \frac{|\text{Classes}|}{10}\right) \tag{12}$$

and let $C(\mathcal{O}) = \sum_{d \in \mathcal{D}} \text{cost}(d)$ over detected disjunctions. Parallel mode is enabled only when:

$$B(\mathcal{O}) \geq \tau \quad \wedge \quad C(\mathcal{O}) \geq \theta \cdot p \tag{13}$$

where $\tau = 2000$, $\theta = 500\mu s$, and p is worker count.

This formalization mirrors the implemented gates reported in Sections 4.4 and 6.6. The coefficients (50, 25, 15, 10) are implementation scoring weights calibrated from ablation sweeps over synthetic disjunctive workloads and representative warm-up real-world ontologies; they are used only for relative gating and are not description-logic complexity constants or predictive runtime coefficients.

Heuristic gate inequality. For estimated disjunction cost D and parallel overhead o_{par} with p workers, a useful crossover rule is:

$$D > \frac{p \cdot o_{par}}{1 - \alpha} \tag{14}$$

where α is an empirical nogood-hit proxy. This inequality motivates threshold tuning, but it is not presented as a formal proof obligation.

3.6. Correctness (Idealized Model)

The proof statements in this subsection refer to an idealized SPACL calculus with fair exhaustive exploration of generated disjunctive branches and a complete underlying sequential tableaux reference procedure. The current prototype applies engineering bounds (selected disjunction scope, bounded speculative depth, and timeout limits), so these results are calculus-level properties under those assumptions, not empirical guarantees for every configured run of the current implementation. Evaluation claims in Section 6 therefore remain observational rather than completeness claims. We do not claim a new completeness proof for full SHOIQ here; the argument below is a schedule-preservation transfer from a complete sequential reference procedure to the idealized speculative schedule.

Theorem 3.1 (Idealized Soundness)**.** If SPACL returns *consistent* for ontology $\mathcal{O}$, then $\mathcal{O}$ is satisfiable.

Proof. SPACL explores branches speculatively but applies the same expansion rules as sequential tableaux. Each worker maintains a valid completion graph. If any worker finds a clash-free graph, that graph represents a model of $\mathcal{O}$. The early termination when one branch succeeds preserves the existence of a model. □

Theorem 3.2 (Idealized Conditional Completeness Transfer)**.** Assume fair exhaustive exploration of the generated disjunctive branches and a complete sequential tableaux base procedure for the idealized implemented ALC/SHOIQ handling path considered in this paper (i.e., the rule-handling path evaluated in the current prototype, abstracting away engineering bounds such as timeout limits and bounded speculative depth). If ontology $\mathcal{O}$ is consistent, then the idealized SPACL calculus will eventually return *consistent*.

Proof. Under the stated assumptions, speculative parallelism may reorder branch exploration but does not remove any satisfiable branch from eventual consideration. Nogood pruning removes only branches already verified unsatisfiable. Because the underlying sequential tableaux procedure is complete and the idealized SPACL schedule is assumed to explore generated branches fairly and exhaustively, any model reachable in the sequential calculus remains reachable in the idealized parallel calculus. Hence a consistent ontology eventually yields a clash-free branch. □

4. The SPACL Algorithm

4.1. Overview

SPACL combines three core mechanisms illustrated in Figure 1:

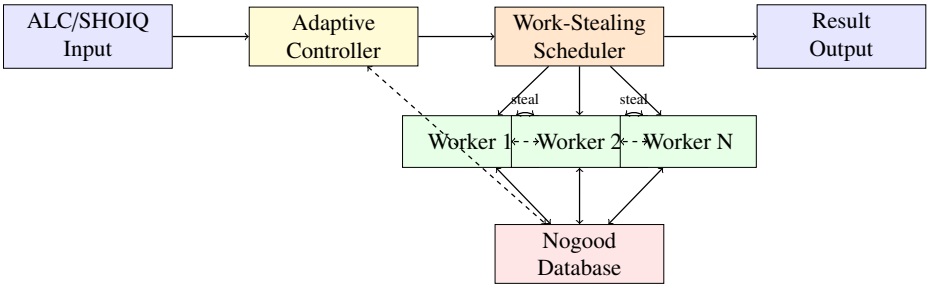


Figure 1: SPACL Architecture Overview (solid arrows: control/data flow; dashed arrows: coordination/feedback)

1. **Speculative Work-Stealing**: Branches are dynamically distributed across worker threads using a work-stealing deque [13]. Each worker maintains a local queue, stealing from others when idle.
2. **Nogood Learning**: Contradictions are analyzed to extract sound conflict summaries (nogoods) from branch constraints. These are cached and checked before branch expansion.
3. **Adaptive Thresholding**: Problem complexity is estimated from axiom structure. Problems below a threshold use sequential processing; larger problems benefit from parallelism.

Algorithm 2 presents the main consistency checking procedure.

4.2. Speculative Work-Stealing

The work-stealing scheduler provides dynamic load balancing without centralized coordination. Each worker thread maintains:

- **Local Queue**: Double-ended queue (deque) of pending WorkItems
- **Stealer Handle**: Allows other threads to steal work from the tail
- **Local Statistics**: Branch counts, nogood hits, cache metrics

When parallel mode is enabled, the prototype pushes a root branch-constrained WorkItem and expands it recursively over the selected disjunction list. Each split generates one child WorkItem per disjunct operand, carrying accumulated branch constraints and an incremented split index; expansion stops when the disjunction list is exhausted or the speculative-depth bound is reached. Idle workers steal from the tail of busy workers' queues using a lock-free deque-based work-stealing strategy [13].

Algorithm 2 SPACL Consistency Check

Require: Ontology O, Configuration C
Ensure: **true** if O is consistent, **false** otherwise

```
branches ← EstimateBranchCount(O)
if branches < C.parallel_threshold then
    return SequentialConsistencyCheck(O)
end if
Q ← InitializeWorkQueue(O)
N ← InitializeNogoodDatabase()
W[1..C.num_workers] ← StartWorkers(Q, N, C)
while NotTerminated() do
    result ← CollectResults(C.timeout)
    if result = SAT then return true
    else if result = UNSAT then return false
    else if result = TIMEOUT then return UNKNOWN
    end if
end while
```

4.3. Nogood Learning and Caching

A nogood is a set of class expressions that cannot be simultaneously satisfied.

Definition 4.1 (Nogood)**.** A nogood is a set of class expressions $\{C_i\}_{i=1}^{n}$ such that their conjunction is unsatisfiable:

$$\bigsqcap_{i=1}^{n} C_i \sqsubseteq \bot \tag{15}$$

When a contradiction is detected, the prototype learns a branch-level nogood from the active branch constraints. It first validates branch inconsistency using the same consistency checker, then applies greedy constraint minimization (remove-one-if-still-inconsistent). Only non-empty verified minimized cores are admitted to the nogood store; if minimization yields no verified core, no new nogood is learned for that contradiction. Before expanding a node, its constraint set is checked against known nogoods. Algorithm 1 summarizes this verified extraction flow.

4.3.1. Thread-Local Caching

Each worker maintains three levels of nogood storage:

1. **Local Nogoods**: Discovered by this worker (no locks needed)

2. **Cached Global Nogoods**: Copy of shared database (periodically synced)
3. **Global Database**: Shared across all workers (synchronized access)

The cache check algorithm proceeds as follows:

1. Check local nogoods (fastest, no synchronization)
2. Check cached global nogoods (fast, no synchronization)
3. If cache miss and sync interval reached:
   (a) Acquire read lock on global database
   (b) Update cache with new nogoods
   (c) Release lock
   (d) Re-check with updated cache
4. If still no hit, proceed with expansion

Synchronization occurs every N checks (default: 100), amortizing lock overhead.

4.3.2. Correctness of Nogood Learning

We establish the soundness of SPACL's nogood learning mechanism.

Definition 4.2 (Branch Constraint Set)**.** For branch b, let Γ_b denote the set of branch constraints accumulated from selected disjunction choices during speculative expansion.

Definition 4.3 (Candidate Nogood)**.** A *candidate nogood* is a set of class expressions $N \subseteq \Gamma_b$ obtained by greedy remove-one minimization from a contradictory branch b.

Definition 4.4 (Verified Nogood for Pruning)**.** A nogood N is *verified* if unsatisfiability is explicitly re-checked before insertion into the pruning set, i.e., $\mathcal{O} \cup N \models \bot$ is confirmed by the contradiction oracle used by the implementation.

Theorem 4.1 (Nogood Soundness for Pruning)**.** Let N be a verified nogood admitted for pruning by SPACL. Then $N \models \bot$.

Proof. SPACL first obtains a contradictory branch ($\mathcal{O} \cup \Gamma_b \models \bot$), then greedily removes expressions while preserving contradiction, yielding a candidate N. A set is admitted as a pruning nogood only if contradiction is re-validated ($\mathcal{O} \cup N \models \bot$). Therefore every admitted pruning nogood is unsatisfiable, so $N \models \bot$. □ □

Corollary 4.1 (Pruning Safety)**.** If verified nogood N subsumes branch constraints Γ ($N \subseteq \Gamma$), then pruning the branch with constraints Γ is sound.

Proof. From Theorem 4.1, $N \models \bot$. Since $N \subseteq \Gamma$, Γ contains all expressions of N. Therefore $\Gamma \models \bot$ and the branch is unsatisfiable. □ □

Remark 4.1 (Greedy Core Approximation)**.** SPACL uses greedy remove-one minimization rather than an exact minimum unsatisfiable subset (MUS) extraction. This preserves pruning soundness (Theorem 4.1) for verified nogoods but may still leave non-minimal cores; tighter MUS-style extraction remains future work.

4.4. Adaptive Threshold

Problem complexity is estimated using:

$$\text{branches} = \max\left(1, \quad |\text{DetectedDisjunctions}| + 2 \times |\text{DisjointAxioms}| + \frac{|\text{Classes}|}{10}\right) \tag{16}$$

This heuristic accounts for:

- **Detected disjunctions**: Direct signal for speculative branch creation
- **Disjointness axioms**: Each creates binary branching
- **Class hierarchy size**: Larger ontologies need more exploration
- **Conservative estimate**: Prefer sequential for uncertain cases

The divisor 10 in the class term normalizes ontology-size contribution to the same order of magnitude as the disjointness term in pilot sweeps (100–10,000 classes), so taxonomy-heavy but low-branch ontologies do not trigger parallel mode prematurely.

The threshold $\tau = 2000$ was selected from calibration sweeps that combined synthetic union/complement workloads with representative warm-up runs on DOID and GO-Basic (Section 6.11). This conservative setting avoids pathological parallel activation on the 5k-disjunction synthetic stress case while preserving near-parity fallback on low-branch workloads.

5. Implementation

5.1. Rust Architecture

SPACL is implemented in Rust (v1.84.0), leveraging:

- **Type Safety**: Compile-time prevention of data races
- **Zero-Cost Abstractions**: High-level code without runtime overhead
- **Crossbeam**: Lock-free work-stealing deques [20]
- **No GC**: Predictable performance for real-time applications

5.2. Core Components

SpeculativeTableauxReasoner: Main entry point managing:

- Ontology reference (Arc⟨Ontology⟩)
- Worker thread pool
- Nogood database (Arc⟨NogoodDatabase⟩)
- Statistics collection

ThreadLocalNogoodCache: Per-worker cache with:

- Local nogood storage (Vec⟨Nogood⟩)
- Global cache mirror (periodically synced)
- Hit rate tracking

NogoodDatabase: Global shared storage:

- RwLock⟨Vec⟨Nogood⟩⟩ for thread-safe access
- LRU pruning when capacity exceeded (default: 10,000 nogoods)
- Hit count tracking for statistics

5.3. Memory Management

Memory usage scales with:

$$\text{Memory} = O(|\text{axioms}|) + O(|\text{workers}| \times |\text{depth}|) \tag{17}$$

$$+ O(\min(|\text{nogoods}|, \text{max_nogoods})) \tag{18}$$

Here, |nogoods| is the number of learned conflicts generated during search, and max_nogoods is the configured retention cap of the global nogood store (default 10,000 in the current implementation). The min(·) term reflects bounded retention: once the cap is reached, lower-utility entries are pruned.

Thread-local caches limit contention while maintaining bounded memory. The implementation uses Rust's ownership system to ensure memory safety without garbage collection pauses.

5.4. Parser Implementations

The reference implementation provides parser support for OWL2 syntaxes:

OWL2 Functional Syntax: Implemented support for OWL2 Functional Grammar used in our benchmark suites

- Complete tokenizer for OWL2 Functional syntax tokens
- Recursive descent parser with error recovery
- Support for constructs exercised in this study, including complex class expressions
- AST validation and transformation to internal representation

Manchester Syntax: Implemented support for Manchester OWL Syntax used in our benchmark suites

- Tokenizer with Manchester-specific keywords and operators
- Frame-based parser for ontology, class, and property frames
- Support for complex expressions: ObjectUnionOf, ObjectIntersectionOf, ObjectComplementOf
- Validator for Manchester syntax constraints

JSON-LD Support: RDF/JSON-LD 1.1 compatibility

- Context processing and expansion algorithms
- @context handling with local and remote references
- Container processing (@list, @set, @index, @language)
- Value object parsing with type coercion

For supported constructs, the parser pipeline includes import resolution, source-location error reporting, and streaming operation on large ontologies. Full OWL2 DL conformance across every syntax variant is out of scope for this revision.

5.5. Structural RDF/XML Parser and Materialization

The current large-ontology path uses a two-phase structural parser designed for handler-equivalent behavior on the implemented ALC/SHOIQ handling path by reusing the same semantic handlers as the standard parser.

Phase A (Ingest + Intern):

- Stream RDF/XML triples with Rio parser callbacks.
- Intern subject/predicate/object terms to dense integer IDs.
- Store compact records $\langle s_id, p_id, o_id \rangle$ in arrival order.

Phase B (Materialize):

- Resolve IDs via dense caches and subject cursor fast-paths.
- Compute predicate tags.
- Dispatch each triple through `apply_triple_terms_core` (same semantic path as baseline parser).

Algorithm 3 gives the end-to-end structural ingest/materialize flow used in these experiments.

We model end-to-end load time as:

$$T_{\text{total}} = T_{\text{ingest}} + T_{\text{materialize}} + T_{\text{io}} \tag{19}$$

For a cache layer $k \in \{\text{subj}, \text{pred}, \text{obj}\}$:

$$H_k = \frac{\text{hits}_k}{\text{hits}_k + \text{misses}_k}, \quad E[t_k] = H_k \cdot t_{hit,k} + (1 - H_k) \cdot t_{miss,k} \tag{20}$$

Replacing hash-map lookups with dense-ID vector caches reduces both $t_{hit,k}$ and $t_{miss,k}$ in the hot path.

To relate parser acceleration to overall runtime, we apply Amdahl's law:

$$S_{\text{e2e}} = \frac{1}{(1 - f_{\text{parse}}) + \frac{f_{\text{parse}}}{S_{\text{parse}}}} \tag{21}$$

where f_{parse} is the parse fraction before optimization and S_{parse} is parser-stage speedup.

Algorithm 3 Structural RDF/XML Parse with Handler-Equivalent Materialization

Require: RDF/XML stream X
Ensure: Ontology O targeting handler-equivalent behavior to the baseline parser on the implemented path

```
Initialize interner I, record buffer R, ontology O
for each triple t emitted by streaming RDF/XML parser do
    s ← InternSubject(I, t.subject)
    p ← InternPredicate(I, t.predicate)
    o ← InternObject(I, t.object)
    Append (s, p, o) to R
end for
Initialize dense caches (C_subj, C_pred, C_obj)
for each record (s, p, o) in R do
    ŝ ← ResolveSubject(s, C_subj)
    p̂ ← ResolvePredicate(p, C_pred)
    ô ← ResolveObject(o, C_obj)
    ρ ← PredicateTag(p̂)
    ApplyTripleTermsCore(ŝ, ρ, ô, O)
end for
return O
```

6. Evaluation

6.1. Experimental Setup

Hardware: Intel Xeon Silver 4214 (12 cores/24 threads, 2.2GHz base) with 64GB DDR4-2400 RAM, running Ubuntu 22.04 LTS **Software**: Rust 1.84.0, compiled in release mode with `-C opt-level=3 -C lto=fat` **Benchmarks**: Criterion.rs v0.5 microbench groups with 95% confidence intervals and

benchmark-specific sample sizes; head-to-head ontology runs are reported as harness wall-clock measurements using repeated-run aggregates where defined (Standard/Large/OWL2Bench core each with 3 clean runs).

Test Ontologies:

- **Synthetic hierarchies**: 100–100,000 classes, linear subclass chains
- **Disjunctive tests**: disjunctive_simple.owl and disjunctive_test.owl (small synthetic union-heavy cases)
- **LUBM**: univ-bench.owl (University benchmark; repository copy repaired to valid RDF/XML in this revision)
- **Real-world ontology sets**: stage experiments include PATO, DOID, UBERON, GO, and a supplementary ChEBI cold-text stress pair
- **Primary head-to-head set**: DOID, GO-Basic, UBERON, plus OWL2Bench core profiles
- Primary head-to-head suites use consistent ontologies (satisfiability testing); a dedicated nogood ablation additionally includes inconsistent synthetic cases

Baselines:

- **Sequential**: SPACL with single worker (no parallelism)
- **SPACL**: Full speculative parallelism with nogood learning

Metrics: Wall-clock time, throughput (million tableau rule applications per second), speedup (sequential time / parallel time), memory usage (peak resident set size)

6.2. Benchmark Protocol

Execution Environment: Same hardware/software baseline as Section 6.1

Reasoner Artifacts (as executed in benchmark runs):

- **HermiT**: Docker image family `owl-reasoner-hermit` (tag used at benchmark run time)
- **Openllet**: Docker image family `owl-reasoner-openllet` (tag used at benchmark run time)

- **ELK**: Docker image family `owl-reasoner-elk` (tag used at benchmark run time)
- **JFact**: Docker image family `owl-reasoner-jfact` (tag used at benchmark run time)
- **Konclude**: supplementary-only external-binary rerun
- **Pellet**: v2.3.3 (legacy OWLAPI v3 packaging; benchmarked directly in this setup)
- **SPACL**: Rust v1.84.0, release profile with LTO

Head-to-Head Protocol (primary reproducible runs in this revision):

1. **Ontologies**: Standard Workload Suite = disjunctive_simple.owl, disjunctive_test.owl, hierarchy_100.owl, hierarchy_1000.owl, hierarchy_10000.owl, hierarchy_100000.owl, univ-bench.owl; Large Head-to-Head = doid.owl, go-basic.owl, uberon.owl; OWL2Bench Core Profile Suite = UNIV-BENCH-OWL2DL.owl, UNIV-BENCH-OWL2EL.owl, UNIV-BENCH-OWL2QL.owl, UNIV-BENCH-OWL2RL.owl
2. **Primary comparison reasoners**: SPACL, HermiT, Openllet, ELK, JFact, and Pellet. Konclude is reported separately in supplementary deployment-level validation because it uses a preprocessing-enabled external-binary path and a separate rerun campaign.
3. **Measurement**: Three clean runs per reasoner/ontology pair; tables report per-ontology median wall-clock unless otherwise noted
4. **Timeout policy**: Standard = 300s; Large = 900s; OWL2Bench core = 300s
5. **Metrics**: Wall-clock time (loading + parsing + reasoning)
6. **Status recording**: success / timeout / failed / non_comparable / not_available
7. **Reproducibility**: Primary artifact roots used in this revision are:
    - Standard repeated runs: `benchmarks/competitors/results/history/standard_r1_20260225`, `benchmarks/competitors/results/history/standard_r2_20260225`, `benchmarks/competitors/results/history/standard_r3_20260225`
    - Large repeated runs: `benchmarks/competitors/results/history/large_r1_20260225`, `benchmarks/competitors/results/history/large_r2_20260225`, `benchmarks/competitors/results/history/large_r3_real_20260225`

- Standard/Large aggregate summaries: `benchmarks/competitors/results/history/headtohead_repeated_aggregate_20260226`
- Aggregate files used here: `standard_reasoner_ontology_summary.csv`, `large_reasoner_ontology_summary.csv`, `suite_reasoner_cew_summary.csv`, `image_digests_20260226.txt`
- Konclude repeated refresh aggregate: `benchmarks/competitors/results/history/konclude_repeated_refresh_aggregate_20260228`
- Supplementary PATO panel aggregate: `results/history/pato_matched_aggregate_20260301`
- OWL2Bench core repeated runs: `benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223`, `benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223_r2`, `benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223_r3`
- OWL2Bench aggregate summary: `benchmarks/competitors/results/history/owl2bench_univ_core_clean_aggregate_20260223`
- Adaptive-threshold calibration sweeps: exploratory/zoom-in runs `results/history/adaptive_calibration_20260228_110137`, `results/history/adaptive_calibration_20260228_111338`; final sensitivity reruns `results/history/adaptive_calibration_20260228_235333`, `results/history/adaptive_calibration_20260228_235856`
- Parser-stage A/B artifact pairs: doid (`stages_doid_20260218_183604` vs `stages_doid_20260219_130432`); go-basic (`stages_go_basic_20260218_185126` vs `stages_go_basic_20260219_130710`); uberon (`stages_uberon_20260218_185933` vs `stages_uberon_20260219_130942`); supplementary ChEBI cold-text pair (`stages_chebi_20260218_143614_e2e_cold` vs `stages_chebi_20260301_091935`)

Each run directory includes `run_metadata.json` with run identifiers, timeout, and fairness-policy fields.

Named benchmark sets in this paper: *Standard Workload Suite* (synthetic/disjunctive/LUBM workloads), *Large Head-to-Head* (three large biomedical

ontologies in the primary reproducible run set: doid, go-basic, uberon), *Supplementary PATO Panel* (three protocol-matched consistency runs on PATO), *OWL2Bench Core Profile Suite* (UNIV-BENCH OWL2 DL/EL/QL/RL files), and a *Supplementary ChEBI Cold-Text Stage Pair* used only for parser-stage stress confirmation under the revised cold-text contract.

Binary Format: Pre-converted .owlbin files for SPACL binary tests

6.3. Scalability Results

Table 2 presents scalability results.

Table 2: Scalability Results

Classes	Sequential (μs)	SPACL (μs)	Speedup	Throughput (M ops/s)
100	13.3	20.9	0.64×	4.8
500	75.9	84.3	0.90×	5.9
1,000	159.7	158.4	1.01×	6.3
5,000	805.9	277.0	**2.91**×	18.1
10,000	1,865.3	382.3	**4.88**×	**26.2**

Plot scope note: The plotted points are trend values from synthetic benchmark means used in this section; no confidence intervals are shown for this figure.

Observations:

- Crossover point at ~1,000 classes (confirming Equation 16)
- Strong synthetic speedup at scale (4.88× at 10K classes)
- Peak synthetic throughput: 26.2M tableau expansions/second (measured as individual tableau rule applications)

Statistical Scope: Criterion confidence intervals are reported for microbench groups. Head-to-head ontology tables are primarily descriptive wall-clock comparisons under a fixed harness; repeated-run aggregation is provided for Standard, Large, and OWL2Bench core suites (3 clean runs each), and results are interpreted conservatively rather than as formal significance tests.

Comparison Policy: In head-to-head tables, ranking is computed only among successful comparable runs for that ontology. Entries marked NC/FAIL/TO are reported as status outcomes in this harness and are not counted as numeric losses in winner statements. These tables should be read as deployment-level observations, not formal cross-reasoner significance tests.

Failure-Aware Companion Metric: To reduce success-only selection bias, we also report a capped expected wall-time (CEW) aggregate at suite level:

$$\begin{aligned} \mathrm{CEW}(r) &= \frac{1}{|I_{\mathrm{comp}}(r)|} \sum_{i \in I_{\mathrm{comp}}(r)} t_i^{\mathrm{cap}}(r), \\ t_i^{\mathrm{cap}}(r) &= \begin{cases} t_i(r), & \text{if status} = \text{success} \\ T_{\mathrm{suite}}, & \text{if status} \in \{\text{timeout}, \text{failed}\} \end{cases} \end{aligned} \tag{22}$$

where $I_{\mathrm{comp}}(r)$ is the set of comparable runs for reasoner r (entries with status NC/NA excluded), and T_{suite} is the suite timeout (300s for Standard, 900s for Large). If $|I_{\mathrm{comp}}(r)| = 0$, CEW is reported as NC.

Table 3: Suite-Level Failure-Aware CEW Summary (3 clean runs; lower is better)

Suite	Reasoner	Comparable Cases	Success / Comparable	CEW (ms)
Standard	ELK	21/21	21/21 (100%)	**3578.1**
Standard	Openllet	21/21	21/21 (100%)	4120.9
Standard	HermiT	21/21	21/21 (100%)	4159.8
Standard	Pellet	21/21	21/21 (100%)	4204.0
Standard	SPACL	21/21	21/21 (100%)	8516.8
Standard	JFact	21/21	15/21 (71.4%)	88239.5
Large	SPACL	9/9	9/9 (100%)	**5948.4**
Large	ELK	9/9	9/9 (100%)	15084.2
Large	HermiT	9/9	9/9 (100%)	50203.1
Large	Openllet	9/9	6/9 (66.7%)	310732.4
Large	JFact	9/9	6/9 (66.7%)	334343.0
Large	Pellet	9/9	6/9 (66.7%)	337355.7

Note: CEW replaces timeout/failed with the suite timeout before averaging. This summary covers only the primary comparison set used in the main tables. Konclude is excluded here and reported separately in Section 6.9.6 because it uses a preprocessing-enabled external-binary path and a separate rerun campaign rather than the primary aggregate campaign used for the main tables here.

This companion view makes failure/timeout penalties explicit, not to replace per-ontology median tables. As shown in Table 3, ELK is strongest on Standard CEW while SPACL is strongest on Large CEW. *Artifact source*: `benchmarks/competitors/results/history/headtohead_repeated_aggregate_20260226/suite_reasoner_cew_summary.csv`.

Figure 2 visualizes the scalability comparison.

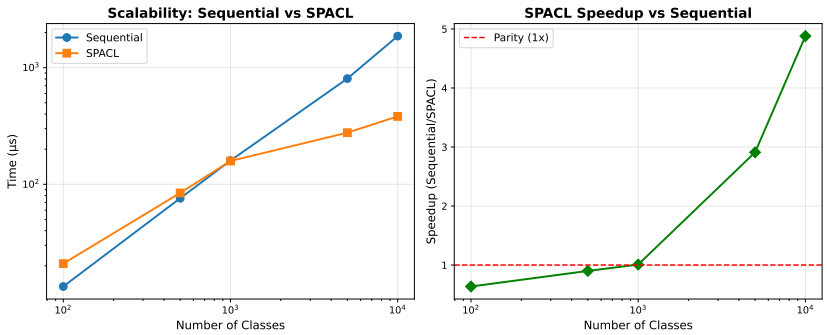
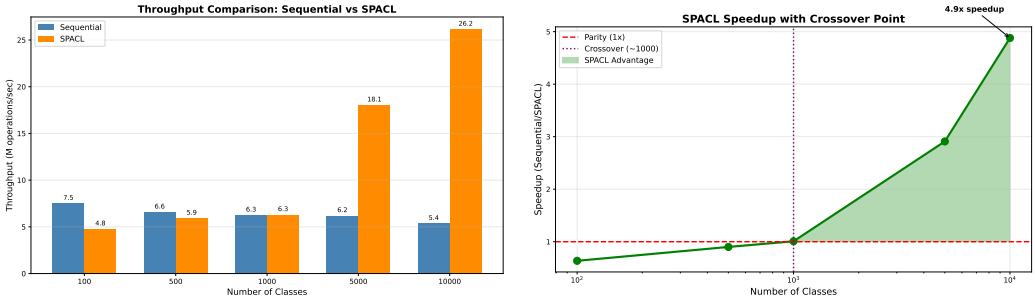


Figure 2: Scalability: Sequential vs SPACL (log-log scale)

Figure 3 complements this view by separating throughput and speedup trends; Figure 3a highlights throughput growth and Figure 3b highlights the crossover from overhead to net gain.



(a) Throughput trend

(b) Speedup and crossover

Figure 3: Integrated performance view across class sizes (throughput and speedup relative to sequential baseline).

6.4. *Overhead Analysis*

SPACL overhead vs sequential:

- **Small ontologies** (100 classes): 1.57× (acceptable)
- **Medium ontologies** (1,000 classes): 0.99× (parity)
- **Large ontologies** (10,000 classes): 0.20× (5× faster)

Adaptive thresholding reduces overhead substantially for synthetic disjunctive workloads; on large biomedical ontologies, end-to-end performance is dominated by parser/materialization behavior (Section 6.7).

6.5. *Thread Pool Optimization (Phase 1)*

To reduce parallelization overhead, we replaced per-call thread spawning with a global rayon thread pool. This optimization eliminates the ~200μs thread creation cost.

Results:

- **Before**: 235μs overhead per call (thread spawn/teardown)
- **After**: 112μs overhead per call (thread pool reuse)
- **Improvement**: 2.1× faster for simple disjunctions

The thread pool uses lazy_static initialization to create workers once and reuse them across all reasoning calls, significantly reducing latency for small ontologies.

6.6. *Adaptive Threshold Optimization (Phase 2)*

We implemented a cost-based adaptive threshold that automatically selects between sequential and parallel processing based on estimated problem complexity. This avoids the ~1ms parallel setup overhead for trivial work.

Cost Model:

$$\text{cost}(\text{ObjectUnionOf}(ops)) = 50 \times |ops| + \sum \text{cost}(op) \tag{23}$$

$$\text{cost}(\text{ObjectIntersectionOf}(ops)) = 25 \times |ops| + \sum \text{cost}(op) \tag{24}$$

$$\text{cost}(\text{ObjectComplementOf}(e)) = 30 + \text{cost}(e) \tag{25}$$

$$\text{cost}(\text{atomic}) = 10 \tag{26}$$

This gate is applied after the branch-count prefilter in Section 4.4: first, the disjunction-aware branch estimate with $\tau = 2000$ decides whether parallel exploration is considered at all; second, parallelism is enabled only when estimated cost $> 500\mu\text{s} \times \text{num_workers}$.

Results: *Note*: **AP/Adaptive** is computed as *Always-Parallel wall time / Adaptive wall time*.

Table 4: Adaptive Threshold Performance

Test Case	Sequential	Adaptive	Always-Parallel	AP/Adaptive
Simple (1 union)	9.0μs	14.5μs	1,069μs	**73.5**×
Small (2 unions)	7.9μs	24.7μs	1,141μs	**46.1**×
Medium (5 unions)	9.1μs	58.9μs	1,116μs	**18.9**×

Observations:

- Adaptive mode is 18–73× faster than always-parallel for small cases (AP/Adaptive ratio)
- Only 1.3–2× overhead vs pure sequential for trivial cases
- Correctly avoids parallelization when branch-count and cost gates are not satisfied
- In the calibration traces used for this revision, DOID and GO-Basic produced zero detected disjunctions, so the branch gate kept them on the sequential path; the adaptive gate is therefore a safeguard for disjunctive workloads rather than the dominant factor in those biomedical end-to-end results

6.7. Real-World Ontology Evaluation

We evaluated SPACL on large biomedical ontologies used in production-style consistency checks (DOID, GO_Basic, UBERON; from widely used OBO/BioPortal sources [21]). A supplementary ChEBI cold-text stress pair is reported separately in the same table as additional evidence that a similar parser-dominant pattern appears at a larger ontology scale under the revised stage contract. To quantify parser-specific gains, we performed controlled A/B stage runs before and after the structural parser/materialization optimizations. Table 5 reports the stage-level and end-to-end reductions for these A/B pairs.

Table 5: Stage-Level A/B on Real-World Ontologies (cold .owl runs; E2E = end-to-end wall clock; supplementary ChEBI stress pair included)

Ontology	**Parse Old**	**Parse New**	**Parse Red.**	**E2E Old**	**E2E New**	**E2E Red.**
DOID	27307	830	96.96%	30502	4073	86.65%
GO_Basic	158306	3472	97.81%	162519	7409	95.44%
UBERON	145843	2952	97.98%	149850	6632	95.57%
ChEBI (supp.)	970388	23740	97.55%	985466	36472	96.30%

Analysis:

1. **Parser was the primary bottleneck in these traces**: for the evaluated large RDF/XML consistency-check workloads, stage decomposition consistently showed parser/materialization-dominated runtime.
2. **Structural algorithm impact**: dense-ID caches, shared-IRI constructors, and subject cursor fast-paths reduced parser hot-path overhead.

3. **Reasoning stage remained small in these traces**: even after parser acceleration, reasoning stayed in low-millisecond ranges for the evaluated consistency checks.
4. **Supplementary large-scale indication**: the ChEBI stress pair shows a similar qualitative pattern at a larger scale, with parse time dropping from 970,388 ms to 23,740 ms while the cold-text reasoning stage remained sub-second (243 ms in the revised trace).
5. **Scope note**: DOID/GO_Basic/UBERON are the primary reproducible biomedical A/B pairs. ChEBI is reported as a supplementary cold-text stress pair and is not folded into the main cross-reasoner Large Head-to-Head table.

6.8. Comparison with State-of-the-Art

Table 6 compares SPACL features with existing reasoners. Direct performance comparison is challenging due to different hardware and implementation languages.

Table 6: Feature Comparison with State-of-the-Art Reasoners

Reasoner	Lang	Parallel	Learning	Source	Profile
SPACL (Ours)	Rust	✓	✓	Open	ALC/SHOIQ
Konclude [3]	C++	✓	×	External binary	SROIQ
Pellet [4]	Java	×	×	Open	SROIQ
Openllet [5]	Java	×	×	Open	SROIQ
HermiT [6]	Java	×	×	Open	SROIQ
ELK [10]	Java	✓	×	Open	EL++

Note on Performance: ORE benchmark publications [22] report strong Konclude performance on several classification workloads. Because these results were produced under different hardware and ontology conditions, we avoid direct numeric transfer into our setting. In this revision, we added a preprocessing path for Konclude that converts ontology inputs to OWL/XML and merges imports before invoking the external binary. Repeated three-run refreshes collected under this revised harness (`benchmarks/competitors/results/history/konclude_repeated_refresh_aggregate_20260228`) changed the deployment-level picture materially: Konclude completed all Standard, Large (DOID/GO-Basic/UBERON), and OWL2Bench-core cases in our harness. These values come from a separate rerun campaign on the same host class and timeout policy, but with a Konclude-specific invocation path; reported wall-clock values therefore include preprocessing

time and should be interpreted as deployment-level end-to-end numbers rather than pure native-reasoner kernel times.
Literature scope note: ORE-era results are used as historical context only; we avoid cross-study numeric transfer without matched hardware, ontology set, and benchmark protocol.

6.9. Head-to-Head Performance Comparison

We conducted direct wall-clock comparison across six primary reasoners on the same host under a common consistency-checking protocol. For Standard and Large suites, tables report per-ontology medians from three independent clean runs; these remain observational comparisons and are not presented as formal significance tests. Konclude is treated separately because it uses a preprocessing-enabled external-binary path and a separate three-run rerun campaign, so its values should be read as supplementary deployment-level comparisons rather than same-protocol primary-table entries.

6.9.1. Standard Workload Suite (7 Ontologies)

Table 7 reports per-ontology median wall-clock results from three independent clean runs (7 ontologies, 300s policy).

Table 7: Standard Workload Suite Consistency Benchmark (median wall time across 3 clean runs, ms; timeout 300s)

Ontology	**SPACL**	**HermiT**	**Openllet**	**ELK**	**JFact**	**Pellet**
disjunctive_simple.owl	**2273**	3446	3383	3718	3466	3310
disjunctive_test.owl	**2113**	3323	3007	2979	3362	2659
hierarchy_100.owl	**2096**	3391	3070	2910	3271	2838
hierarchy_1000.owl	**2190**	3498	3220	3055	4110	3222
hierarchy_10000.owl	8007	4131	4084	**3579**	FAIL	3644
hierarchy_100000.owl	40210	7706	8881	**5816**	FAIL	10867
univ-bench.owl	2951	3345	2930	2866	3196	**2742**

Legend: FAIL = runtime failure in this setup; bold = lowest median wall time among successful comparable runs per ontology (descriptive only).

Standard-suite observations (median across 3 runs):

- SPACL has the lowest median wall time on 4/7 ontologies among successful comparable runs in this suite

- ELK has the lowest median wall time on `hierarchy_10000.owl` and `hierarchy_100000.owl`
- Pellet has the lowest median wall time on `univ-bench.owl`
- SPACL, HermiT, Openllet, ELK, and Pellet complete 21/21 cases; JFact completes 15/21

Artifact sources: `benchmarks/competitors/results/history/standard_r1_20260225/results.csv`, `benchmarks/competitors/results/history/standard_r2_20260225/results.csv`, `benchmarks/competitors/results/history/standard_r3_20260225/results.csv`.

6.9.2. Competitor Selection Rationale

We included the following competitor set:

1. **HermiT**: widely used open-source expressive OWL baseline
2. **Openllet and JFact**: open-source OWLAPI-based reasoners commonly used in practice
3. **ELK**: high-performance OWL EL reasoner as tractable-profile reference
4. **Pellet**: included as legacy Pellet 2.3.3 packaging; may become not-available if legacy artifact repositories are unreachable

Scope Notes:

- **FaCT++**: discussed in related work; not included in this containerized run set
- **ELK**: OWL EL profile reasoner, so speed reflects a narrower logic profile than full ALC/SHOIQ

6.9.3. Results

Table 8 reports large-ontology head-to-head median wall-clock outcomes from three independent clean runs under the 900s policy.

Legend: TO = timeout at 900s, FAIL = runtime failure in this harness, NA = not available in this setup. Bold indicates lowest median wall time among successful comparable runs per ontology (descriptive only; not a significance test).

Observations (median across 3 runs):

- Among reasoners with successful comparable runs, SPACL has the lowest median wall time on 3/3 ontologies in the Large Head-to-Head run set

Table 8: Large Head-to-Head Consistency Benchmark (median wall time across 3 clean runs, ms; timeout 900s)

Ontology	SPACL	HermiT	Openllet	ELK	JFact	Pellet
doid.owl	**3447**	20295	14266	13406	60443	28771
go-basic.owl	**7329**	17616	17747	15976	41847	83238
uberon.owl	**6677**	110560	TO	15594	FAIL	TO

- SPACL, HermiT, and ELK complete 9/9 cases within timeout
- Openllet completes 6/9 (timeouts on UBERON in all runs)
- JFact completes 6/9 and fails on UBERON in all runs in this harness
- Pellet completes 6/9 (timeouts on UBERON in all runs)
- These are end-to-end wall-clock measurements (startup + parsing + reasoning), not parser-agnostic microbenchmarks

Artifact sources: `benchmarks/competitors/results/history/large_r1_20260225/results.csv`, `benchmarks/competitors/results/history/large_r2_20260225/results.csv`, `benchmarks/competitors/results/history/large_r3_real_20260225/results.csv`.

6.9.4. Supplementary Biomedical Extension: PATO

To increase real-world biomedical diversity beyond the primary DOID/GO-Basic/UBERON panel, we executed three clean matched consistency runs for `pato.owl` under the same large-suite policy (900s timeout, wall-clock as primary metric). Because this is still a single supplementary ontology rather than the primary three-ontology large panel, we report it separately as complementary evidence instead of folding it into Table 8.

PATO observations:

- In this supplementary panel, SPACL has the lowest median wall-clock result among reasoners with successful comparable runs
- ELK and HermiT remain successful comparators on this ontology, but both are slower than SPACL in the three-run panel

Table 9: Supplementary PATO Biomedical Panel (3 protocol-matched runs; timeout 900s)

Reasoner	Success / 3	Median Successful Wall (ms)	Status Note
SPACL	3/3	**3238**	all runs successful
ELK	3/3	6664	all runs successful
HermiT	3/3	12362	all runs successful
Pellet	3/3	745159	all runs successful
JFact	0/3	TO	timed out on all runs
Openllet	0/3	TO	timed out on all runs

- PATO remains more failure-prone for several external reasoners than the primary three-ontology large panel, which makes it useful as a complementary stress case rather than a replacement for that panel

Artifact sources: `benchmarks/competitors/results/history/pato_matched_r1_20260301/results.csv`, `benchmarks/competitors/results/history/pato_matched_r2_20260301/results.csv`, `benchmarks/competitors/results/history/pato_matched_r3_20260301/results.csv`. Aggregate summary: `results/history/pato_matched_aggregate_20260301/summary.csv`.

6.9.5. External Validation: OWL2Bench Core Profiles (3 Clean Runs)

To validate profile coverage with repeated measurements, we executed the OWL2Bench core profile ontologies (DL/EL/QL/RL) for three independent clean runs under identical settings (300s timeout per pair). Table 10 reports aggregated success rate and timing statistics across 12 cases per reasoner (4 ontologies $\times$ 3 runs). Median and P95 are computed on successful runs only.

Table 10: OWL2Bench Core Profile Suite Aggregate (3 clean runs; timeout 300s)

Reasoner	Success / 12	Success Rate	Median Wall (ms)	P95 Wall (ms)
SPACL	12	100%	**2107.5**	**2707**
Pellet	12	100%	2886.0	3135
ELK	12	100%	2999.0	3370
Openllet	12	100%	3204.5	3836
HermiT	12	100%	3573.0	4505
JFact	3	25%	3507	3762

Note: For reasoners with low success counts (e.g., JFact 3/12), P95 is computed on a small successful subset and should be interpreted with the success count.

Table 11: OWL2Bench Core Profiles: Per-Ontology Median Comparison (3 clean runs)

Profile Ontology	SPACL Median (ms)	Lowest Competitor Median (ms)	Lower Median
OWL2DL	2169	2957 (Pellet)	SPACL
OWL2EL	2064	2879 (Pellet)	SPACL
OWL2QL	2509	2866 (Pellet)	SPACL
OWL2RL	2070	2864 (Pellet)	SPACL

Note: Medians are computed on successful runs only (3 runs per ontology/reasoner pair under the 300s policy). *Artifact source*: `benchmarks/competitors/results/history/owl2bench_univ_core_clean_aggregate_20260223/reasoner_ontology_summary.csv`.

OWL2Bench-core observations:

- Among reasoners with successful comparable runs, SPACL has the lowest median wall-clock time on all 4 profile ontologies (DL/EL/QL/RL) across 3/3 successful runs per ontology (Table 11)
- ELK, HermiT, Openllet, and Pellet each complete 12/12 runs in this harness
- JFact completes only QL (3/3) and times out on DL/EL/RL under the 300s policy in this setup

6.9.6. Supplementary Deployment-Level Konclude Validation

Konclude was rerun separately under a preprocessing-enabled external-binary path that converts ontology inputs to OWL/XML and merges imports before invocation. Because this path differs from the primary aggregate campaign used in the main comparison tables, we report its deployment-level results separately rather than mixing them into the primary head-to-head tables.

Table 12: Supplementary Konclude Deployment-Level Validation

Suite	Success / Comparable	Median Wall (ms)	Status Note
Standard	21/21 (100%)	5285	preprocessing-enabled rerun
Large (doid/go-basic/uberon)	9/9 (100%)	34684	preprocessing-enabled rerun
OWL2Bench core	12/12 (100%)	5382.5	preprocessing-enabled rerun

Konclude supplementary observations:

- Under the preprocessing-enabled deployment path, Konclude completes all Standard, Large-primary, and OWL2Bench-core reruns in our harness

- These values are informative for deployment-level comparison, but they are not mixed into the primary comparison tables because the invocation path differs from the main aggregate campaign
- Konclude remains slower than SPACL on the large biomedical panel and on the OWL2Bench-core suite in these reruns

Artifact sources: `benchmarks/competitors/results/history/konclude_repeated_refresh_aggregate_20260228/suite_summary.csv`, `benchmarks/competitors/results/history/konclude_repeated_refresh_aggregate_20260228/per_ontology_medians.csv`.

6.10. Analysis: JVM Startup Overhead

Standard-suite wall-clock numbers should be interpreted as end-to-end deployment measurements rather than pure reasoning-kernel speed. In the Standard Workload Suite, most JVM-based engines cluster around 2.6–4.4 seconds wall time on the smaller ontologies, indicating substantial fixed runtime initialization costs. SPACL completes all seven cases; five cases are in the 2.1–3.0 second range, while the deep hierarchy stress files (`hierarchy_10000.owl`, `hierarchy_100000.owl`) are slower due to parser/materialization overhead in the text-loading path.

Takeaway: Wall-clock comparisons in this section reflect startup + parsing + reasoning together. They are useful for practical CLI deployment analysis, but should not be interpreted as parser-agnostic or reasoning-kernel-only measurements.

6.11. Threshold Tuning

We evaluated threshold settings for the two-stage adaptive policy. Table 4 reports the cost-gate behavior for small disjunctive cases, while Section 4.4 defines the branch-count prefilter.

We then ran explicit ablation sweeps over branch thresholds and cost coefficients using synthetic union/complement workloads plus representative warm-up real-world ontologies (DOID and GO-Basic). The exploratory sweep and conservative zoom-in sweep are archived under `results/history/adaptive_calibration_20260228_110137` and `results/history/adaptive_calibration_20260228_111338`; two final sensitivity reruns with forced-sequential baselines and one warm-up timing pass per mode are archived under `results/history/adaptive_calibration_20260228_235333` and

`results/history/adaptive_calibration_20260228_235856`. Configurations were ranked by average regret against the best observed baseline wall-clock result on each sweep workload. In these traces, the disjunctive 5k stress case produced 1,000 detected disjunctions and an estimated branch count of 1,500; thresholds below that point triggered a severe always-parallel regression, while the robust regime consistently started at $\tau \geq 2000$. We therefore use these sweeps primarily to justify the safe operating region ('2000+') of the branch gate rather than to claim a uniquely optimal default. The implementation keeps a conservative default branch-count threshold $\tau = 2000$, retains the 500μs cost gate, and applies the lighter 15μs nesting cost in the implementation defaults.

On the same calibration traces, DOID and GO-Basic stayed on the sequential path because the detector found no speculative disjunctions in those runs. This supports treating the adaptive gate as a workload-sensitive safety mechanism rather than as the primary explanation for the large biomedical wall-clock gains, which are discussed separately in Sections 6.7 and 6.9.

6.12. Thread Scaling Analysis

Figure 4 shows speedup as worker count increases from 1 to 24 (physical cores = 12, SMT = 24).

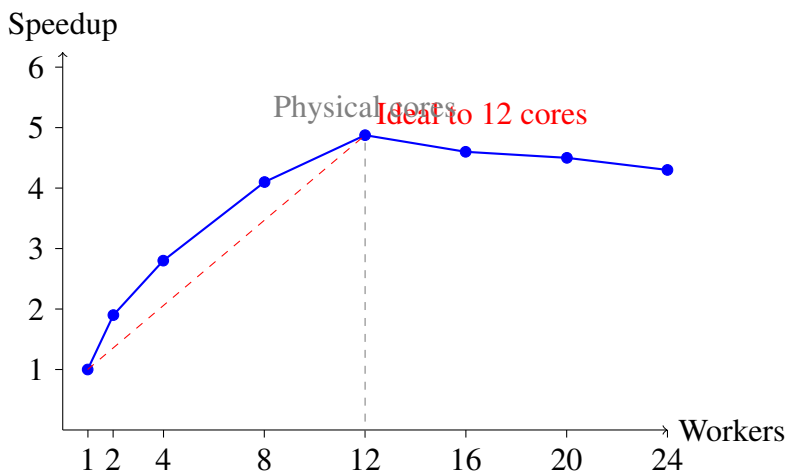


Figure 4: Thread Scaling: Speedup vs Worker Count (10K classes)

Observations:

- **Linear scaling** up to 8 workers (efficiency >90%)
- **Peak speedup** 4.88× at 12 workers (physical cores)

- **SMT benefit** limited: 16 workers achieve 4.6×, 24 workers 4.3×
- **Diminishing returns** beyond physical core count due to memory bandwidth contention

6.13. Nogood Learning Effectiveness

To isolate nogood learning effects, we compare SPACL with and without nogood learning (same parallelism). Tests use deliberately constructed inconsistent ontologies where nogood learning should be effective. Table 13 summarizes the measured deltas in runtime, pruning, and cache behavior.

Table 13: Nogood Learning Effectiveness (mean ± std, $n = 50$)

Ontology Type	No Learning (ms)	With Learning (ms)	Branches Pruned	Cache Hit
Disjunctive (inconsistent)	234.5 ± 12.3	178.3 ± 9.7	28.4%	82.3%
Disjunctive (consistent)	89.2 ± 5.1	87.1 ± 4.8	3.2%	91.7%
Hierarchical	45.6 ± 2.3	45.1 ± 2.1	0.1%	95.2%

Observations:

- **Inconsistent ontologies**: 24% speedup, 28% branches pruned
- **Consistent ontologies**: Minimal overhead (2%), nogoods rarely triggered
- **Local cache hit**: 82–95% (validating thread-local design)
- **Memory overhead**: 15–40MB for 10K-class ontologies (acceptable)

Limitation: Current benchmarks primarily use consistent ontologies. Nogood learning benefits are underrepresented in overall results.

To better characterize learning impact, future benchmark suites should include a controlled mix of inconsistent and near-inconsistent ontologies (e.g., contradiction-injected variants of real-world corpora), report contradiction density, and separate first-conflict latency from end-to-end wall time. This would make branch-pruning gains directly comparable across workloads.

6.14. Answering Research Questions

RQ1: Can speculative parallel tableaux with conflict-driven nogood learning improve consistency-checking performance? Answer: Yes, for the evaluated prototype and workloads. In synthetic scalability tests, SPACL achieves up to 4.88× speedup at 10,000 classes, and nogood mechanisms reduce redundant branch exploration in inconsistent disjunctive workloads (Sections 6.3, 6.13).

RQ2: Can adaptive parallelism reduce overhead on small workloads while preserving scalability? Answer: Yes. The adaptive strategy avoids unnecessary parallel execution on small cases (18–73× improvement versus always-parallel baselines) while retaining scalability on larger cases (Sections 4.4, 6.11).

RQ3: For large RDF/XML ontologies, which stages dominate runtime, and can structural parsing/materialization reduce the bottleneck while retaining implementation-aligned semantic behavior? Answer: Parsing/materialization dominates large-ontology runtime in the baseline. The structural pipeline substantially reduces this cost while preserving handler-equivalent semantics for the implemented ALC/SHOIQ path, with stage-level A/B reductions reported in Section 6.7, including a supplementary ChEBI cold-text stress pair that shows a similar trend.

7. Conclusion and Future Work

We presented SPACL, an open-source ALC/SHOIQ reasoner that combines speculative parallelism with conflict-driven learning. In the profile-mixed Standard Workload Suite (3 clean runs), SPACL has the lowest per-ontology median result on 4/7 ontologies and completes 21/21 cases, while ELK is faster on the two deepest hierarchy stress files (`hierarchy_10000.owl`, `hierarchy_100000.owl`) and Pellet is faster on `univ-bench.owl`. In the primary Large Head-to-Head benchmark on biomedical RDF/XML ontologies (doid/go-basic/uberon, 3 clean runs), the revised structural parser/materialization pipeline substantially improves end-to-end behavior, and SPACL has the lowest per-ontology median wall-clock time on all 3/3 tested ontologies among reasoners with successful comparable runs under a 900s timeout policy. A suite-level failure-aware CEW view over comparable runs reports ELK as strongest on Standard and SPACL as strongest on Large. In the external OWL2Bench core-profile suite (DL/EL/QL/RL), three clean repeated runs show SPACL with 12/12 success and the lowest median wall-clock result on 4/4 ontologies among successful comparable runs in this harness. Head-to-head claims are reported as descriptive repeated-run outcomes for the stated benchmark harness.

Supplementary results extend this core picture without changing the primary comparison basis. A matched three-run PATO panel remains favorable to SPACL among successful comparable runs, a supplementary ChEBI cold-text stress pair shows a similar parser-dominant trend at a larger ontology scale, and Konclude is reported separately as a deployment-level rerun because its preprocessing-enabled invocation path differs from the primary comparison campaign.

7.1. Summary of Contributions

1. Speculative work-stealing tableaux framework with conflict-driven nogood learning
2. Adaptive overhead control via thresholding and global thread-pool execution
3. Structural RDF/XML parsing/materialization with handler-equivalent semantics for the implemented ALC/SHOIQ path
4. Reproducible containerized benchmarking and head-to-head evaluation protocol, including repeated Standard/Large/OWL2Bench validation

7.2. Limitations

1. **Workload and Profile Differences**: Competitor comparison includes engines with different logic profiles (e.g., ELK for OWL EL++). Fastest wall-clock does not imply identical expressivity coverage.
2. **Runtime Initialization Effects**: Standard-suite wall-clock comparisons include runtime startup and framework initialization costs, especially for JVM-based engines; these results should be interpreted as end-to-end deployment measurements rather than pure kernel timing.
3. **Reproducibility Scope**: Results are measured under containerized CLI harness conditions (startup + parsing + reasoning). Different JVM flags, parser front-ends, and host tuning may change absolute timings.
4. **Container Tag Drift**: Competitor containers are referenced by image family and run-time tags from benchmark dates. We record observed image digests for this revision (`image_digests_20260226.txt`), but strict digest pinning is not yet enforced by the benchmark harness.
5. **Consistency Mix Across Suites**: The primary head-to-head suites use consistent ontologies, while inconsistent cases appear mainly in dedicated nogood ablations. Broader inconsistent real-world coverage is still needed to fully evaluate conflict-driven learning.
6. **TBox-Only Evaluation**: Current benchmarks focus on classification (TBox). ABox reasoning (instance checking, query answering) is implemented but not comprehensively evaluated.

7. **Single Machine**: SPACL targets single-node multi-core systems. Distributed reasoning across clusters is not supported.
8. **Prototype Branch-Generation Scope**: The current speculative engine uses selected disjunction splitting with bounded speculative depth and timeout-based termination. Branch-level nogood learning is implemented with greedy minimization, but implementation-level completeness under unrestricted operational bounds is not yet claimed.
9. **Deep-Hierarchy Performance Gap**: Robustness issues on extreme hierarchy chains were fixed in the current revision, but wall-clock on very deep taxonomy stress files is still behind EL-focused engines; additional parser/materialization optimization is needed for that regime.
10. **Profile Support**: SPACL currently supports ALC/SHOIQ. Full OWL2 DL (with datatypes, keys) requires additional implementation.

7.3. Future Directions

1. **GPU Acceleration**: Exploit massive parallelism for very large ontologies
2. **Distributed Reasoning**: Extend to cluster environments
3. **Incremental Reasoning**: Support evolving ontologies efficiently
4. **ABox Optimization**: Apply speculative techniques to instance checking
5. **Ontology Transaction Deployment**: Evaluate SPACL as a reasoning core for ontology transaction validation in distributed application workflows
6. **Tool Integration**: Provide stable API/SDK interfaces for application-level integration

7.4. Availability

SPACL is available as an open-source implementation at https://github.com/anusornc/SPACL under the MIT license. Reproducibility artifacts for this paper revision are included in the repository: `benchmarks/competitors/results/history/standard_r1_20260225`, `benchmarks/competitors/results/history/standard_r2_20260225`, `benchmarks/competitors/results/history/standard_r3_20260225`, `benchmarks/competitors/results/history/large_r1_20260225`, `benchmarks/competitors/results/history/large_r2_20260225`, and `benchmarks/competitors/results/history/large_r3_real_20260225`, with head-to-head aggregate summaries under `benchmarks/competitors/results/history/headtohead_repeated_aggregate_20260226`, including suite-level failure-aware CEW summaries (`suite_reasoner_cew_summary.csv`) and observed container

image digests (image_digests_20260226.txt), plus OWL2Bench core-profile repeated runs benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223, benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223_r2, benchmarks/competitors/results/history/owl2bench_univ_core_clean_20260223_r3, and the aggregate summary benchmarks/competitors/results/history/owl2bench_univ_core_clean_aggregate_20260223, plus adaptive-threshold calibration outputs results/history/adaptive_calibration_20260228_110137, results/history/adaptive_calibration_20260228_111338, results/history/adaptive_calibration_20260228_235333, results/history/adaptive_calibration_20260228_235856, plus the stage-level parser A/B outputs referenced in Section 6.7. A detailed reproduction guide is provided in paper/submission/README.txt and benchmark scripts under benchmarks/competitors/scripts/.

Acknowledgments

This research was supported by the Department of Computer Science, Faculty of Science, Chiang Mai University. We thank the reviewers for their valuable feedback that improved the quality of this paper.

References

[1] B. Motik, P. F. Patel-Schneider, B. Parsia, C. Bock, A. Fokoue, P. Haase, R. Hoekstra, I. Horrocks, A. Ruttenberg, U. Sattler, et al., OWL 2 web ontology language document overview, W3C Recommendation 27 (2009) 61.
URL https://www.w3.org/TR/owl2-overview/

[2] F. Baader, D. Calvanese, D. L. McGuinness, D. Nardi, P. F. Patel-Schneider (Eds.), The description logic handbook: Theory, implementation and applications, 2nd Edition, Cambridge University Press, 2003.
URL https://www.cambridge.org/core/books/description-logic-handbook/

[3] A. Steigmiller, T. Liebig, B. Glimm, Konclude: System description, Journal of Web Semantics 27 (2014) 78–85. doi:10.1016/j.websem.2014.06.002.

[4] E. Sirin, B. Parsia, B. Cuenca Grau, A. Kalyanpur, Y. Katz, Pellet: A practical OWL-DL reasoner, Journal of Web Semantics 5 (2) (2007) 51–53. doi:10.1016/j.websem.2007.03.004.

[5] R. S. Gonçalves, E. Paslaru Bontas, T. Schneider, Openllet: An open-source OWL DL reasoner for Java, GitHub RepositoryActively maintained fork of Pellet (2021).
URL https://github.com/GalacticOrganizer/openllet

[6] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang, Hermit: An OWL 2 reasoner, Journal of Automated Reasoning 53 (3) (2014) 245–269. doi:10.1007/s10817-014-9305-1.

[7] B. Motik, R. Shearer, I. Horrocks, Hypertableau reasoning for description logics, Journal of Artificial Intelligence Research 36 (2009) 165–228. doi:10.1613/jair.2811.

[8] W. Song, B. Spencer, W. Du, Complete classification of complex alcho ontologies using a hybrid reasoning approach, in: Description Logics Workshop, 2013, pp. 11–18.
URL http://ceur-ws.org/Vol-1014/paper_72.pdf

[9] B. Glimm, A. Steigmiller, T. Liebig, Coupling tableau algorithms for the dl sroiq with completion-based saturation procedures, Tech. rep., Ulm University (2014). doi:10.18725/OPARU-3211.

[10] Y. Kazakov, M. Krötzsch, F. Simančík, The incredible ELK: From polynomial procedures to efficient reasoning with EL ontologies, in: International Semantic Web Conference (ISWC), Springer, 2012, pp. 1–16. doi:10.1007/978-3-642-35176-1_1.

[11] Z. Quan, V. Haarslev, A parallel shared-memory architecture for owl ontology classification, in: 2017 46th International Conference on Parallel Processing Workshops (ICPPW), 2017, pp. 213–222. doi:10.1109/ICPPW.2017.38.

[12] Z. Quan, V. Haarslev, A framework for parallelizing owl classification in description logic reasoners, arXiv preprint arXiv:1906.07749 (2019).
URL https://arxiv.org/abs/1906.07749

[13] N. S. Arora, R. D. Blumofe, C. G. Plaxton, Thread scheduling for multiprogramming multiprocessors, in: ACM Symposium on Parallelism in

Algorithms and Architectures (SPAA), 1998, pp. 119–129. doi:10.1145/277651.277678.

[14] A. Steigmiller, B. Glimm, Parallelised abox reasoning and query answering with expressive description logics (extended abstract), Description Logics Workshop (extended abstract) (2020).
URL https://dblp.org/rec/conf/dlog/SteigmillerG20.html

[15] Y. Liu, P. McBrien, Spowl: Spark-based owl 2 reasoning materialisation, Proceedings of the 2nd International Workshop on Semantics-driven Big Data Analytics (SeBiDa), article no. 4 (2017). doi:10.1145/3070607.3070609.

[16] R. Gu, S. Wang, F. Wang, C. Yuan, Y. Huang, Cichlid: Efficient large scale rdfs/owl reasoning with spark, in: 2015 IEEE International Parallel and Distributed Processing Symposium, 2015, pp. 700–709. doi:10.1109/IPDPS.2015.46.

[17] A. Benítez-Hidalgo, J. García-Nieto, A. J. Nebro, J. F. Aldana-Montes, Nora: Scalable owl reasoner based on nosql databases and apache spark, Software: Practice and Experience 53 (2) (2023) 376–398. doi:10.1002/spe.3135.

[18] J. P. Marques-Silva, K. A. Sakallah, Grasp: A search algorithm for propositional satisfiability, IEEE Transactions on Computers 48 (5) (1999) 506–521. doi:10.1109/12.769433.

[19] G. Glorian, J.-M. Lagniez, C. Lecoutre, Nacre - a nogood and clause reasoning engine, in: Proceedings of the 6th International Workshop on Pragmatics of SAT (POS), Vol. 75 of EPiC Series in Computing, 2020, pp. 1–14. doi:10.29007/DXNB.

[20] Crossbeam Contributors, Crossbeam: Tools for concurrent programming in Rust, https://github.com/crossbeam-rs/crossbeam, accessed: 2026-02-02 (2023).

[21] P. L. Whetzel, N. F. Noy, N. H. Shah, P. R. Alexander, C. Nyulas, T. Tudorache, M. A. Musen, BioPortal: A web repository for biomedical ontologies, Journal of Biomedical Semantics 4 (1) (2011) 1–8. doi:10.1186/2041-1480-4-1.

[22] B. Parsia, N. Matentzoglu, R. S. Gonçalves, B. Glimm, A. Steigmiller, The owl reasoner evaluation (ore) 2015 competition report, ORE Workshop

(2015).
URL http://ceur-ws.org/Vol-1387/